



UNIVERSITÀ DI PARMA

DIPARTIMENTO DI SCIENZE MATEMATICHE, FISICHE E INFORMATICHE

Corso di Laurea Triennale in Informatica

Metodologie per l'Orientamento al Pensiero Computazionale: un Caso di Studio basato sull'Algoritmo Heap Sort

*Methodologies for Computational Thinking Orientation: A Case
Study based on the Heap Sort Algorithm*

CANDIDATO:

Kenneth James Nna Minkousse

RELATORE:

Prof. Vincenzo Bonnici

Indice

Introduzione	1
1 Orientamento al Pensiero Computazionale	3
1.1 Cos'è il pensiero computazionale	3
1.1.1 I quattro pilastri del Pensiero Computazionale	3
1.2 Perché insegnarlo: obiettivi educativi e formativi	4
1.3 Metodologie didattiche per l'orientamento	4
1.3.1 Programmazione visuale e attività unplugged	5
1.3.2 Learning-by-doing e problem solving	5
1.3.3 Gamification e Ambienti Interattivi	5
1.3.4 Game-based learning (GBL)	5
1.4 L'algoritmo Heap Sort come strumento didattico	6
1.4.1 Fondamenti teorici dell'Heap	6
1.4.2 Analisi dell'Algoritmo	6
1.4.2.1 Fase 1: Build-Max-Heap (Costruzione)	6
1.4.2.2 Fase 2: Sorting (Ordinamento)	7
1.4.3 Analisi della Complessità (Big O Notation)	7
2 Descrizione e Analisi del Progetto	9
2.1 Obiettivi del Progetto di Tirocinio	9
2.1.1 1. Obiettivi Didattici e Cognitivi	9
2.1.2 2. Obiettivi Tecnici e Architettureali	10
2.1.3 3. Obiettivi di User Experience (Gamification)	10
2.2 Descrizione Dettagliata dell'Applicazione	11
2.2.1 Interfaccia Utente (UI) e Layout	11
2.2.2 Flusso di Interazione (User Journey)	11
2.3 Strumenti e Tecnologie: Analisi Tecnica	12
2.3.1 HTML5: Semantica e Accessibilità	12
2.3.2 CSS3 Avanzato: Grid, Flexbox e Variabili	12
2.3.3 JavaScript (ES6+) Moderno	13
2.4 Analisi Approfondita dei Requisiti	13

2.4.1	Requisiti Funzionali (RF)	13
2.4.2	Requisiti Non Funzionali (RNF)	14
3	Progettazione tramite Diagrammi UML	16
3.1	Diagrammi dei Casi d'Uso	16
3.1.1	Casi d'Uso Principali	16
3.1.2	Dettaglio Casi d'Uso	17
3.1.2.1	UC01 - Selezione Tipo Heap	17
3.1.2.2	UC02 - Inizio Partita	19
3.1.2.3	UC03 - Swap Nodi	20
3.1.2.4	UC04 - Estrazione Radice	22
3.1.2.5	UC05 - Reset Gioco	23
3.1.2.6	UC06 - Vittoria/Sconfitta	24
3.1.3	Matrice di Tracciabilità	25
3.2	Diagramma delle Attività	26
3.2.1	Attività Principali	26
3.2.2	Dettaglio del Flusso Principale (Workflow)	27
3.3	Diagramma delle Classi	28
3.3.1	GamePresenter (Presenter)	29
3.3.2	Heap (Model)	30
3.3.3	Renderer (View)	31
3.3.4	Computer	32
3.3.5	Node	32
3.3.6	Interfaccia Strategy	32
3.4	Diagramma di Sequenza	33
3.4.1	Sequenza Principale del Gioco	34
3.4.1.1	1. Gestione degli Eventi e Selezione (View -> Presenter)	35
3.4.1.2	2. Validazione Logica (Presenter -> Model)	35
3.4.1.3	3. Manipolazione dei Dati e Coerenza (Model)	36
3.4.1.4	4. Riconciliazione della Vista (Presenter -> View)	36
3.4.1.5	5. Verifica Post-Mossa e Automazione (Chain Reaction)	36
3.4.2	Interazione tra Classi - Turno Computer	38
3.4.2.1	1. Orchestrazione Temporale e Innesco (Timer -> Presenter)	38
3.4.2.2	2. Analisi Strategica e Scansione del Modello (Computer -> Heap)	39
3.4.2.3	3. Il Pattern Strategy in Azione (Model -> Strategy)	39

3.4.2.4	4. Esecuzione della Mutazione e Integrità del Modello (Model Execution)	40
3.4.2.5	5. Feedback Loop e Ritorno del Controllo (Computer -> Presenter)	40
3.4.2.6	6. Sincronizzazione della Vista e Rendering Differenziale (View Phase)	40
3.4.2.7	7. Considerazioni Finali sul Flusso Computer	41

Introduzione

L'informatica e il **pensiero computazionale** hanno assunto un ruolo centrale nella formazione moderna, diventando competenze imprescindibili per la comprensione della realtà digitale che ci circonda. Tuttavia, l'apprendimento delle strutture dati e degli algoritmi fondamentali rappresenta spesso uno scoglio significativo per gli studenti universitari e delle scuole superiori. Concetti astratti come la ricorsione, la complessità asintotica o la manipolazione di alberi binari, se affrontati esclusivamente attraverso lo studio teorico sui libri di testo o tramite diagrammi statici, possono risultare ostici, aridi e privi di un riscontro concreto immediato.

Un esempio emblematico di questa difficoltà è rappresentato dall'algoritmo di ordinamento **Heap Sort**. Sebbene sia uno degli algoritmi più eleganti ed efficienti (con una complessità temporale di $O(n \log n)$), la sua comprensione richiede uno sforzo cognitivo notevole: lo studente deve visualizzare mentalmente una struttura dati (l'array) che viene interpretata logica come un'altra (un albero binario quasi completo), mantenendo costantemente traccia delle relazioni matematiche tra gli indici (padre, figlio sinistro, figlio destro). Questa "doppia natura" dell'Heap crea spesso confusione e ostacola l'interiorizzazione profonda del meccanismo di ordinamento.

Questo divario tra teoria astratta e comprensione pratica è il punto di partenza del presente progetto di tirocinio. L'obiettivo primario è stato quello di progettare e sviluppare un'applicazione web interattiva, denominata "**Heap Sort Challenge**", che funga da ponte tra la definizione formale dell'algoritmo e la sua esecuzione dinamica. Sfruttando le moderne tecnologie web (HTML5, CSS3, JavaScript ES6) e adottando un approccio pedagogico basato sulla **Gamification**, il progetto trasforma il processo di apprendimento da un'attività passiva di studio a una sfida attiva e coinvolgente.

A differenza delle tradizionali animazioni algoritmiche (presenti in molti siti didattici), dove l'utente si limita a premere "Play" e osservare le barre muoversi, "Heap Sort Challenge" pone lo studente al centro dell'azione. L'applicazione implementa una modalità "Uomo contro Macchina": l'utente deve competere contro una CPU simulata, prendendo decisioni in tempo rea-

le su quali nodi scambiare per soddisfare le proprietà matematiche del Max-Heap. Questo approccio "*Learning-by-Doing*" costringe l'utente a ragionare come l'algoritmo, valutando le condizioni e prevedendo le conseguenze di ogni scambio, favorendo così un apprendimento più duraturo e profondo.

Dal punto di vista tecnico, il progetto è stato l'occasione per applicare metodologie di ingegneria del software rigorose in un contesto frontend. L'applicazione è stata architettata seguendo il pattern **Model-View-Presenter (MVP)**, garantendo una netta separazione tra la logica algoritmica (il Modello), la rappresentazione visiva nel DOM (la Vista) e la logica di controllo e interazione (il Presenter). Questa scelta non solo ha facilitato lo sviluppo e il testing, ma rappresenta di per sé un caso di studio interessante su come strutturare applicazioni web client-side moderne e manutenibili senza l'ausilio di framework pesanti.

Nel seguito di questo elaborato, il lavoro svolto verrà presentato secondo la seguente strutturazione:

Nel **Capitolo 1** verrà introdotto il contesto teorico del Pensiero Computazionale, analizzando come gli strumenti interattivi possano potenziare le capacità di astrazione, decomposizione e riconoscimento di pattern, competenze chiave per ogni informatico.

Il **Capitolo 2** sarà dedicato all'analisi approfondita dell'algoritmo Heap Sort. Verranno richiamati i fondamenti teorici, le proprietà della struttura dati Heap e la logica matematica che ne governa il funzionamento, fornendo il background necessario per comprendere le scelte implementative.

Nel **Capitolo 3** si affronterà la fase di Progettazione e Design. Verranno illustrati i diagrammi UML che descrivono l'architettura del sistema, le scelte di User Interface (UI) e User Experience (UX) mirate a massimizzare l'usabilità, e la struttura del pattern MVP adottato.

Il **Capitolo 4** entrerà nel dettaglio dell'Implementazione tecnica. Verranno discussi gli aspetti più rilevanti del codice sorgente, come la gestione del ciclo di vita del gioco, l'implementazione della "Intelligenza Artificiale" avversaria e le tecniche CSS utilizzate per garantire animazioni fluide e performanti.

Infine, nel **Capitolo 5**, verranno trattate le Conclusioni, analizzando i risultati ottenuti rispetto agli obiettivi prefissati, discutendo le limitazioni attuali del sistema e proponendo possibili sviluppi futuri per estendere la piattaforma ad altri algoritmi o modalità di gioco.

Capitolo 1

Orientamento al Pensiero Computazionale

1.1 Cos'è il pensiero computazionale

Il pensiero computazionale, termine reso celebre da Jeannette Wing nel 2006, rappresenta "un approccio fondamentale per la risoluzione di problemi, la progettazione di sistemi e la comprensione del comportamento umano che attinge ai concetti fondamentali dell'informatica". Non si tratta semplicemente di "pensare come un computer", che è un mero esecutore di istruzioni, ma piuttosto di pensare con i livelli di astrazione necessari per rendere un problema risolvibile da una macchina. È un processo cognitivo che coinvolge diverse dimensioni chiave:

1.1.1 I quattro pilastri del Pensiero Computazionale

1. **Astrazione:** È la capacità di filtrare i dettagli superflui per concentrarsi sugli aspetti essenziali di un problema. *Esempio nel progetto:* Nel gioco Heap Sort, l'astrazione è rappresentata dal modo in cui i numeri vengono visualizzati come nodi di un albero. I dettagli di basso livello (come i bit che compongono il numero in memoria) sono nascosti; ciò che conta è la relazione logica padre-figlio.
2. **Decomposizione:** Consiste nel suddividere un problema complesso in sotto-problemi più piccoli e gestibili. *Esempio nel progetto:* L'ordinamento di un intero array viene decomposto in due fasi principali (Heapify e Sorting), e a sua volta ogni fase è composta da singole operazioni atomiche di "swap" e "confronto". Lo studente impara che risolvere un problema grande significa risolverne tanti piccoli in sequenza.

3. **Riconoscimento di pattern:** La capacità di identificare somiglianze, tendenze e regolarità nei dati. *Esempio nel progetto:* Riconoscere che ogni sotto-albero deve soddisfare la stessa proprietà (es. Max-Heap) permette di applicare la stessa logica ricorsivamente a tutta la struttura.
4. **Algoritmica:** La definizione di una serie di passi ordinati e precisi (un algoritmo) per raggiungere un obiettivo. Non basta "sapere" la soluzione, bisogna saperla descrivere in modo che sia replicabile.

1.2 Perché insegnarlo: obiettivi educativi e formativi

L'introduzione del pensiero computazionale nelle scuole risponde alla necessità di formare cittadini consapevoli in una società digitale (Digital Literacy). Tuttavia, il valore educativo trascende l'alfabetizzazione informatica:

- **Meta-cognizione:** Imparare a programmare o a simulare algoritmi aiuta gli studenti a riflettere sul proprio modo di pensare ("thinking about thinking").
- **Problem Solving Trasversale:** Le tecniche apprese (decomposizione, astrazione) sono applicabili in matematica, scienze, linguistica e persino nella gestione di progetti quotidiani.
- **Gestione dell'Errore:** In informatica, l'errore (bug) non è un fallimento, ma un'informazione utile per correggere il processo. Questo approccio favorisce una "Growth Mindset" (mentalità di crescita), riducendo l'ansia da prestazione scolastica.
- **Creatività Computazionale:** Fornire gli strumenti per passare da consumatori passivi di tecnologia (fruitori di app) a creatori attivi, capaci di esprimere le proprie idee attraverso il codice o la logica algoritmica.

1.3 Metodologie didattiche per l'orientamento

Per insegnare efficacemente questi concetti, è necessario allontanarsi dalla lezione frontale trasmissiva ("Instructionism") per abbracciare metodologie costruttiviste ("Constructionism" di Seymour Papert).

1.3.1 Programmazione visuale e attività unplugged

La programmazione visuale (es. Scratch, Blockly) riduce il carico cognitivo legato alla sintassi del codice (punti e virgola, parentesi), permettendo di concentrarsi sulla semantica e sulla logica. Le attività *CS Unplugged* (senza computer) utilizzano giochi fisici, carte, corda o movimenti corporei per introdurre concetti informatici. Ad esempio, ordinare una fila di studenti per altezza è un modo fisico per spiegare un algoritmo di sorting. Queste attività rendono l'informatica accessibile anche in contesti con scarse risorse tecnologiche.

1.3.2 Learning-by-doing e problem solving

L'approccio "imparare facendo" è fondamentale. Invece di memorizzare definizioni astratte di "ciclo" o "variabile", gli studenti affrontano problemi reali. La conoscenza emerge come sottoprodotto dell'attività di risoluzione del problema. Nel contesto del nostro progetto, lo studente non "studia" l'Heap Sort, ma lo "esegue", internalizzando le regole attraverso la pratica.

1.3.3 Gamification e Ambienti Interattivi

La *gamification* applica meccaniche di gioco (punti, livelli, classifiche, sfide a tempo) a contesti non ludici per aumentare la motivazione intrinseca ed estrinseca. Gli ambienti interattivi offrono un **feedback immediato**. Nel modello tradizionale (compito in classe), il feedback (voto) arriva giorni dopo l'esecuzione, quando ormai il momento di apprendimento è passato. In un software educativo, se lo studente sbaglia un passaggio, il sistema lo notifica istantaneamente (es. nodo rosso), permettendo una correzione in tempo reale. Questo ciclo di feedback rapido è essenziale per l'apprendimento profondo.

1.3.4 Game-based learning (GBL)

A differenza della gamification che "aggiunge" elementi di gioco a un processo esistente, il *Game-Based Learning* utilizza veri e propri giochi come veicolo principale per l'apprendimento. Il gioco diventa il "libro di testo" interattivo. In "Heap Sort Challenge", il contenuto didattico (l'algoritmo) è intrinseco alle meccaniche di gioco: per vincere, devi necessariamente comprendere l'algoritmo.

1.4 L'algoritmo Heap Sort come strumento didattico

L'Heap Sort è stato scelto come oggetto di questo progetto per la sua ricchezza concettuale. Non è il più semplice (come il Bubble Sort) né sempre il più veloce nella pratica (come il QuickSort), ma offre spunti didattici unici sulla gestione efficiente dei dati.

1.4.1 Fondamenti teorici dell'Heap

Un **Binary Heap** è una struttura dati che può essere vista come un albero binario quasi completo. Un albero si dice "quasi completo" se tutti i livelli sono riempiti completamente, tranne eventualmente l'ultimo, che viene riempito da sinistra a destra. Questa proprietà strutturale è cruciale perché permette di memorizzare l'albero in un array compatto senza l'uso di puntatori espliciti, garantendo un'efficienza spaziale ottimale.

Dato un nodo all'indice i (0-indexed):

- Il figlio sinistro è a: $2i + 1$
- Il figlio destro è a: $2i + 2$
- Il genitore è a: $\lfloor \frac{i-1}{2} \rfloor$

Oltre alla proprietà strutturale, l'Heap deve soddisfare la *Proprietà di Ordine*:

- **Max-Heap:** Ogni nodo è maggiore o uguale ai suoi figli ($A[\text{parent}(i)] \geq A[i]$). Di conseguenza, l'elemento massimo è sempre nella radice.
- **Min-Heap:** Ogni nodo è minore o uguale ai suoi figli ($A[\text{parent}(i)] \leq A[i]$). L'elemento minimo è nella radice.

1.4.2 Analisi dell'Algoritmo

L'algoritmo Heap Sort procede in due fasi distinte, entrambe visualizzate nel software:

1.4.2.1 Fase 1: Build-Max-Heap (Costruzione)

Si parte da un array disordinato. Si interpretano i dati come un albero binario. Partendo dall'ultimo nodo non-foglia (indice $n/2 - 1$) e risalendo fino alla radice (indice 0), si applica la procedura **Heapify** verso il basso.

Questa fase trasforma l'array caotico in un Max-Heap valido. La complessità di questa fase, con un'analisi attenta, risulta essere lineare: $O(n)$.

1.4.2.2 Fase 2: Sorting (Ordinamento)

Una volta costruito l'Heap:

1. Si scambia la radice (massimo attuale) con l'ultimo elemento dell'heap ($A[n - 1]$).
2. Si riduce la dimensione considerata dell'heap di 1 (l'elemento massimo è ora "congelato" nella sua posizione finale corretta).
3. La nuova radice probabilmente viola la proprietà di Max-Heap, quindi si esegue **Heapify** sulla radice per far "affondare" il valore nella posizione corretta.
4. Si ripete fino a quando l'heap ha dimensione 1.

1.4.3 Analisi della Complessità (Big O Notation)

Uno degli obiettivi formativi avanzati è far comprendere agli studenti l'efficienza degli algoritmi.

- **Tempo:** La procedura **Heapify** su un nodo richiede tempo proporzionale all'altezza dell'albero, ovvero $O(\log n)$. Poiché nella fase di sorting eseguiamo **Heapify** n volte, la complessità totale è $O(n \log n)$. Questo vale per tutti i casi:
 - **Best Case:** $O(n \log n)$ (o $O(n)$ se le chiavi sono uguali, ma generalmente domina il logaritmo).
 - **Average Case:** $O(n \log n)$.
 - **Worst Case:** $O(n \log n)$. Questa è una proprietà molto importante rispetto al QuickSort, che nel caso peggiore degrada a $O(n^2)$.
- **Spazio:** L'Heap Sort è un algoritmo **in-place**. Richiede solo una quantità costante di memoria ausiliaria $O(1)$ per lo scambio delle variabili, a differenza del MergeSort che richiede $O(n)$ spazio extra per gli array di appoggio.

- **Stabilità:** È un algoritmo **instabile**. Lo scambio di elementi distanti può alterare l'ordine relativo di elementi con chiavi uguali. Questo è un dettaglio tecnico importante che viene spesso discusso in corsi universitari.

Algoritmo 1 Max-Heapify (Versione Ricorsiva)

Require: Array A , indice i

Ensure: Sottoalbero radicato in i conforme alla proprietà di Max-Heap

```

1:  $left \leftarrow 2i + 1$ 
2:  $right \leftarrow 2i + 2$ 
3:  $largest \leftarrow i$ 
4: if  $left < A.heapSize \wedge A[left] > A[largest]$  then
5:    $largest \leftarrow left$ 
6: if  $right < A.heapSize \wedge A[right] > A[largest]$  then
7:    $largest \leftarrow right$ 
8: if  $largest \neq i$  then
9:   Swap  $A[i]$  with  $A[largest]$ 
10:  Heapify( $A, largest$ )

```

Algoritmo 2 Heap Sort Completo

Require: Array A da ordinare

Ensure: Array A ordinato in senso crescente

```

1: Fase 1: Costruzione dell'Heap
2: for  $i \leftarrow \lfloor A.length/2 \rfloor - 1$  downto 0 do
3:   Max-Heapify( $A, i$ )
4: Fase 2: Estrazione e Ordinamento
5: for  $i \leftarrow A.length - 1$  downto 1 do
6:   Swap  $A[0]$  with  $A[i]$ 
7:    $A.heapSize \leftarrow A.heapSize - 1$ 
8:   Max-Heapify( $A, 0$ )

```

Capitolo 2

Descrizione e Analisi del Progetto

2.1 Obiettivi del Progetto di Tirocinio

Il progetto "Heap Sort Challenge" nasce dalla convergenza di due esigenze: una didattica (migliorare l'apprendimento degli algoritmi) e una tecnica (sfruttare le potenzialità del web moderno). L'obiettivo primario è decostruire la complessità dell'Heap Sort, trasformandolo da "concetto astratto da memorizzare" a "sistema dinamico da esplorare".

Gli obiettivi sono stati categorizzati secondo tre direttrici principali:

2.1.1 1. Obiettivi Didattici e Cognitivi

Basandosi sulla tassonomia di Bloom rivisitata, il software mira a supportare diversi livelli di apprendimento:

- **Ricordare e Capire (Livelli base):** Attraverso la visualizzazione, lo studente memorizza la struttura dell'albero e comprende le relazioni padre-figlio.
- **Applicare (Livello intermedio):** Nella modalità di gioco, lo studente deve applicare attivamente le regole di swap per risolvere le violazioni di heap.
- **Analizzare e Valutare (Livelli avanzati):** Confrontando la propria strategia con quella della CPU, lo studente valuta l'efficienza delle proprie mosse e analizza gli errori commessi.

Specificamente, ci si attende che alla fine dell'utilizzo lo studente sappia:

1. Identificare visivamente una violazione della proprietà di Max-Heap o Min-Heap.

2. Prevedere l'effetto di uno scambio sulla struttura globale dell'albero.
3. Spiegare perché l'estrazione avviene sempre dalla radice.

2.1.2 2. Obiettivi Tecnici e Architetture

La sfida tecnica non riguardava solo la "traduzione" dell'algoritmo in codice, ma la creazione di un sistema software robusto e manutenibile.

- **Architettura Disaccoppiata:** Implementare rigorosamente il pattern MVP (Model-View-Presenter). Il *Model* (logica dell'heap) non deve sapere nulla dell'HTML; la *View* (DOM) non deve contenere logica di business; il *Presenter* funge da direttore d'orchestra. Questo permette, ad esempio, di cambiare completamente l'interfaccia grafica senza toccare una riga dell'algoritmo.
- **Performance delle Animazioni:** Garantire i 60fps (frame per secondo) durante le animazioni di swap. Questo ha richiesto l'uso attento di proprietà CSS performanti (come `transform` e `opacity`) evitando quelle che causano il "reflow" della pagina (come `top`, `left`, `margin`).
- **Responsive Design:** L'albero binario deve adattarsi a schermi di diverse dimensioni senza sovrapposizioni dei nodi, una sfida non banale data la natura espansiva delle strutture ad albero.

2.1.3 3. Obiettivi di User Experience (Gamification)

Per mantenere alto l'engagement ("coinvolgimento"), sono state applicate le euristiche di Nielsen e principi di Gamification:

- **Visibilità dello Stato:** L'utente deve sempre sapere di chi è il turno e qual è lo stato corrente dell'heap.
- **Feedback Immediato:** Ogni azione deve avere una reazione. Un click valido "accende" il nodo; un click invalido lo fa "tremare" (shake animation) in rosso.
- **Challenge (Sfida):** La presenza della CPU non è cosmetica. L'algoritmo avversario è stato tarato per essere veloce ma non istantaneo, creando una "race condition" psicologica che spinge l'utente a pensare più in fretta.

2.2 Descrizione Dettagliata dell'Applicazione

L'applicazione si presenta come una "Single Page Application" (SPA), dove l'intero flusso di gioco avviene senza ricaricamenti di pagina, garantendo un'esperienza fluida simile a un'app nativa.

2.2.1 Interfaccia Utente (UI) e Layout

L'interfaccia è divisa logicamente in tre macro-aree:

1. **Header e Controlli (Top Bar):** Qui risiedono i controlli globali. Un selettore (dropdown) permette di scegliere tra "Min Heap" e "Max Heap". Un pulsante prominente "Start Game" avvia la simulazione. Un display del tempo e dei turni tiene traccia della partita.
2. **Area di Gioco (Main Stage):** Lo schermo è spaccato verticalmente (Split Screen):
 - **Sinistra (Player Zone):** È l'area interattiva. L'utente vede il proprio heap. I nodi sono cerchi cliccabili, connessi da linee (SVG o bordi CSS) che rappresentano gli archi dell'albero. Sotto l'albero, viene visualizzata la rappresentazione lineare (Array View) che si aggiorna in sincronia, fondamentale per far capire la corrispondenza Albero $\leftrightarrow$ Array.
 - **Destra (CPU Zone):** È l'area speculare ma non interattiva. Qui l'utente osserva l'algoritmo eseguito dalla macchina. I nodi si muovono da soli, offrendo un esempio visivo corretto da imitare.
3. **Area di Feedback (Overlay/Toast):** Messaggi temporanei appaiono per notificare eventi critici: "Mossa non valida!", "Heap Costruito!", "Hai Vinto!".

2.2.2 Flusso di Interazione (User Journey)

Immaginiamo una sessione tipica di uno studente:

1. **Ingresso:** Lo studente apre la pagina. L'albero è vuoto. Seleziona "Max Heap".
2. **Setup:** Preme "Start". L'algoritmo genera 15 numeri casuali. I nodi appaiono sullo schermo con un'animazione di entrata ("pop-in").
3. **Fase 1 - Heapify:**

- Lo studente esamina l'albero. Nota che il nodo 40 è figlio del nodo 20. In un Max-Heap, il padre deve essere maggiore.
- Clicca sul 40 (si illumina). Clicca sul 20.
- I nodi si scambiano di posto con un'animazione fluida. L'array sottostante si aggiorna: [..., 40, ...] prende il posto di [..., 20, ...].
- Contemporaneamente, la CPU fa la sua mossa sulla destra.

4. Fase 2 - Extract:

- Una volta che tutto l'albero è verde (proprietà soddisfatta), lo studente clicca sulla radice (il massimo).
- La radice vola via verso l'array ordinato finale. L'ultimo nodo dell'heap prende il posto della radice (diventando temporaneamente viola per indicare la violazione).
- Il gioco riprende con la fase di Heapify sul nuovo, più piccolo, heap.

5. **Endgame:** Quando l'ultimo nodo è ordinato, compare la schermata di riepilogo: "Vittoria! Tempo: 45s - Mosse: 24".

2.3 Strumenti e Tecnologie: Analisi Tecnica

La scelta dello stack tecnologico non è stata casuale, ma guidata dal requisito di *Universalità* e *Longevità*.

2.3.1 HTML5: Semantica e Accessibilità

Non abbiamo usato 'div' generici per tutto. L'uso di tag come '<main>', '<section>', '<header>' e '<button>' garantisce che l'applicazione sia interpretabile correttamente anche dagli Screen Reader per utenti ipovedenti. Ogni nodo dell'albero ha attributi 'aria-label' e 'data-value' per esporre il proprio contenuto sia agli strumenti di assistenza che agli script di test.

2.3.2 CSS3 Avanzato: Grid, Flexbox e Variabili

Il layout dell'albero binario è una delle parti più complesse.

- **CSS Grid:** È stato utilizzato per disegnare la struttura dell'albero. Ogni livello dell'albero corrisponde a una riga della griglia; le colonne

sono calcolate per centrare perfettamente ogni nodo rispetto ai suoi figli.

- **CSS Variables (Custom Properties):** Definire colori come `--primary-color: #4CAF50` o `--node-size: 50px` in un unico punto (il blocco `:root`) ha permesso di implementare facilmente temi diversi e di mantenere la coerenza visiva.
- **Hardware Acceleration:** Per le animazioni di movimento (`transform: translate`), il browser delega il calcolo alla GPU invece che alla CPU, garantendo fluidità anche su dispositivi mobili meno potenti.

2.3.3 JavaScript (ES6+) Moderno

Il codice è scritto aderendo allo standard ECMAScript 2015+.

- **Classi e OOP:** Il paradigma a oggetti mappa perfettamente la realtà del problema. `class Node` ha proprietà `value` e `element` (riff. DOM). `class Heap` contiene la logica matematica.
- **Async/Await e Promises:** La gestione del tempo è cruciale. L'animazione dura 500ms. Il codice JavaScript non può "bloccarsi" aspettando l'animazione. Abbiamo usato le `Promise` per dire al codice: "Aspetta che la transizione CSS finisca prima di aggiornare logico i dati".
- **Modularity:** L'uso di `import/export` nativi permette di avere file piccoli e focalizzati (`Heap.js`, `Renderer.js`, `Game.js`) invece di un unico file monolitico ("Spaghetti Code"), facilitando enormemente il debugging.

2.4 Analisi Approfondita dei Requisiti

2.4.1 Requisiti Funzionali (RF)

Dettagliamo ulteriormente il comportamento atteso:

RF01 - Configurazione Partita Il sistema deve impedire l'avvio della partita se non è stata selezionata una modalità valida. Il cambio di modalità a partita in corso deve resettare lo stato corrente per evitare incoerenze (es. un Min-Heap parzialmente ordinato trattato come Max-Heap).

RF02 - Generazione Heap e Rendering L'algoritmo di generazione deve produrre numeri pseudo-casuali in un range leggibile (es. 1-100) per evitare nodi con numeri troppo lunghi che uscirebbero dai cerchi grafici. La visualizzazione deve supportare almeno 4 livelli di profondità (fino a 15 nodi).

RF03 - Interazione e Validazione Mosse Il core del gioco. Il sistema deve intercettare il click.

- *Primo Click*: Il nodo viene evidenziato (stato "Selected").
- *Secondo Click*:
 - Se è lo stesso nodo: Deseleziona.
 - Se è un nodo non adiacente (non padre/figlio): Errore visivo, reset selezione.
 - Se è adiacente ma non viola la proprietà di heap (swap inutile): Errore visivo ("Mossa non necessaria!").
 - Se è adiacente e viola la proprietà: Esegui swap, aggiorna punteggio.

RF04 - Intelligenza Artificiale (CPU) La CPU non deve "barare" risolvendo istantaneamente l'array. Deve simulare un "tempo di pensiero" e un "tempo di azione" per essere umanamente seguibile. Deve implementare una coda di azioni per non sovrapporre le animazioni.

RF05 - Rilevamento Fine Gioco e Vittoria Il sistema deve verificare la condizione di vittoria ad ogni completamento di una mossa. La vittoria è definita come: `heapSize == 0` E array è ordinato correttamente.

2.4.2 Requisiti Non Funzionali (RNF)

- **RNF01 - Performance**: Il Time-to-Interactive (TTI) all'avvio deve essere sotto 1 secondo. Le animazioni non devono scendere sotto i 30fps su mobile.
- **RNF02 - Usabilità**: Il design deve seguire il principio di "Affordance": ciò che è cliccabile deve sembrare cliccabile (cursor pointer, hover effects). I colori devono essere distinguibili anche da utenti daltonici (uso di simboli o pattern oltre al colore rosso/verde).
- **RNF03 - Portabilità**: Il layout Grid deve "collassare" in modo grazioso su tablet (Responsive Breakpoints).

- **RNF04 - Manutenibilità:** Il codice deve essere commentato seguendo lo standard JSDoc. Non devono esserci "Magic Numbers" nel codice; tutti i parametri di configurazione (velocità animazione, colori, dimensione heap) devono essere in costanti o file di configurazione.

Capitolo 3

Progettazione tramite Diagrammi UML

3.1 Diagrammi dei Casi d'Uso

Il Diagramma dei Casi d'Uso (Use Case Diagram) rappresenta le funzionalità del sistema dal punto di vista degli attori che vi interagiscono. Nel contesto di "Heap Sort Challenge", l'attore principale è lo **Studente** (o Utente Generico), mentre il sistema reagisce alle sue azioni.

3.1.1 Casi d'Uso Principali

Di seguito vengono descritti i casi d'uso principali identificati durante l'analisi dei requisiti funzionali, mappando le interazioni chiave dell'applicazione.

3.1.2 Dettaglio Casi d'Uso

3.1.2.1 UC01 - Selezione Tipo Heap



Figura 3.1: Diagramma dei Casi d'Uso per UC01 - Selezione Tipo Heap

Di seguito vengono riportati i dettagli formali del caso d'uso:

ID Caso d'Uso	UC01
Nome	Selezione Modalità Heap (Min/Max)
Attori Primari	Studente
Descrizione	L'utente configura l'applicazione per operare in modalità Min-Heap (minimo in radice) o Max-Heap (massimo in radice).
Precondizioni	• L'applicazione è avviata. • L'interfaccia grafica è caricata correttamente.
Postcondizioni	• La variabile di stato <code>currentHeapType</code> è aggiornata. • L'area di gioco viene resettata per coerenza. • Il feedback visivo della modalità è aggiornato.

Scenario Principale	1. L'utente accede al menu di configurazione. 2. Il sistema mostra le opzioni disponibili: "Max Heap" e "Min Heap". 3. L'utente seleziona l'opzione desiderata. 4. Il sistema cattura l'evento di cambio selezione (<i>change event</i>). 5. Il Presenter invoca il metodo di reset del Model passando la nuova modalità. 6. La View aggiorna il titolo della pagina e i colori tematici (es. Rosso per Max, Blu per Min).
Scenario Alternativo	Cambio durante una partita in corso: 1. L'utente cambia modalità mentre sono presenti nodi nell'area di gioco. 2. Il sistema rileva che lo stato non è vuoto. 3. Il sistema forza un "Reset" completo (vedi UC05) prima di applicare la nuova modalità, per evitare che un Max-Heap parziale venga valutato con le regole del Min-Heap.

3.1.2.2 UC02 - Inizio Partita

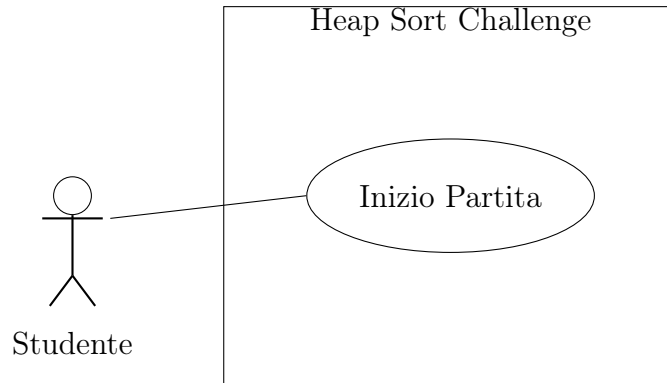


Figura 3.2: Diagramma dei Casi d'Uso per UC02 - Inizio Partita

ID Caso d'Uso	UC02
Nome	Inizio Partita (Start Game)
Attori Primari	Studente
Descrizione	L'utente avvia una nuova sessione di gioco. Il sistema genera un array di nodi casuali (o accetta input manuale) e li visualizza in una struttura ad albero binario.
Precondizioni	• L'utente ha selezionato la modalità di Heap (UC01). • L'applicazione è pronta (stato idle).
Postcondizioni	• L'albero è popolato con nodi. • I controlli di gioco (Swap, Step Successivo) sono abilitati. • Il timer è avviato o resettato.

Scenario Principale	1. L'utente preme il pulsante "Start" o inserisce una lista di numeri e conferma. 2. Il sistema valida l'input (se presente). 3. Il Model crea l'istanza dell'Heap iniziale. 4. La View renderizza i nodi nell'area di canvas. 5. Il sistema abilita le interazioni sui nodi.
Scenario Alternativo	Input non valido: 1. L'utente inserisce caratteri non numerici. 2. Il sistema mostra un messaggio di errore. 3. La partita non inizia.

3.1.2.3 UC03 - Swap Nodi

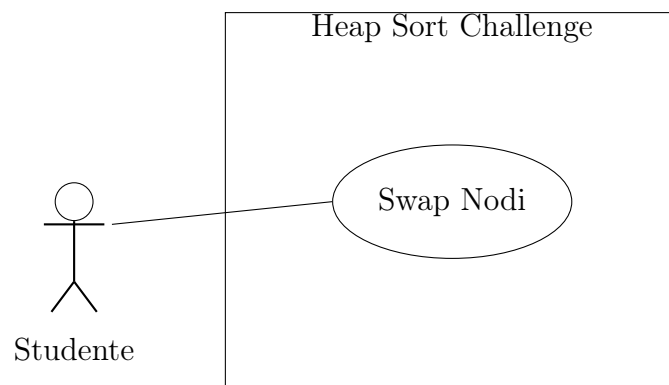


Figura 3.3: Diagramma dei Casi d'Uso per UC03 - Swap Nodi

ID Caso d'Uso	UC03
Nome	Swap Nodi (Scambio)
Attori Primari	Studente

Descrizione	L'utente seleziona due nodi (normalmente un genitore e un figlio) per scambiare la posizione nell'albero.
Precondizioni	• Partita in corso. • L'albero contiene almeno due nodi.
Postcondizioni	• I valori dei due nodi sono scambiati. • Il contatore delle mosse viene incrementato.
Scenario Principale	1. L'utente clicca sul primo nodo (Source). 2. Il sistema evidenzia il nodo selezionato. 3. L'utente clicca sul secondo nodo (Target). 4. Il sistema esegue lo scambio visivo. 5. Il sistema verifica la correttezza della mossa (opzionale, per feedback immediato).
Scenario Alternativo	Deselezione: 1. L'utente clicca nuovamente sul nodo selezionato. 2. La selezione viene annullata.

3.1.2.4 UC04 - Estrazione Radice

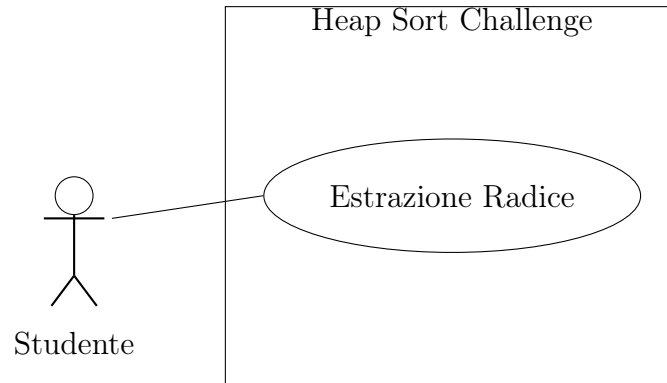


Figura 3.4: Diagramma dei Casi d'Uso per UC04 - Estrazione Radice

ID Caso d'Uso	UC04
Nome	Estrazione della Radice (Extract Root)
Attori Primari	Studente
Descrizione	Quando la proprietà di Heap è soddisfatta (es. Max-Heap completo), l'utente estrae la radice per spostarla nell'array ordinato.
Precondizioni	• L'albero rispetta la proprietà di Heap (tutti i padri $\geq$ figli). • È il turno dell'utente.
Postcondizioni	• La radice viene rimossa e spostata nell'array ordinato. • L'ultimo nodo foglia prende il posto della radice. • La dimensione dell'Heap diminuisce di 1.

Scenario Principale	1. L'utente clicca sul pulsante "Estrai Radice" (o sulla radice stessa). 2. Il sistema verifica che l'albero sia un Heap valido. 3. Il nodo radice si sposta nell'area "Ordinati". 4. L'ultimo nodo diventa la nuova radice.
Scenario Alternativo	Heap non valido: 1. L'utente tenta l'estrazione ma l'albero non è ordinato correttamente. 2. Il sistema mostra un errore: "La proprietà di Heap non è soddisfatta".

3.1.2.5 UC05 - Reset Gioco

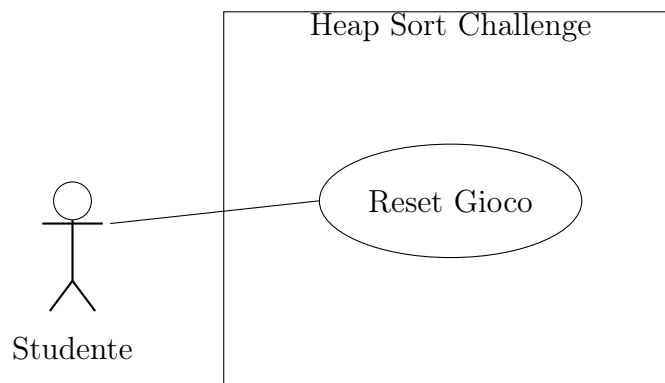


Figura 3.5: Diagramma dei Casi d'Uso per UC05 - Reset Gioco

ID Caso d'Uso	UC05
Nome	Reset Gioco
Attori Primari	Studente
Descrizione	L'utente interrompe la partita corrente e riporta l'applicazione allo stato iniziale.

Precondizioni	Nessuna.
Postcondizioni	• L'albero viene svuotato. • L'array di input viene pulito. • Il timer e i contatori vengono azzerati.
Scenario Principale	1. L'utente preme il pulsante "Reset". 2. Il sistema chiede conferma (opzionale). 3. Il sistema ripristina lo stato iniziale.

3.1.2.6 UC06 - Vittoria/Sconfitta

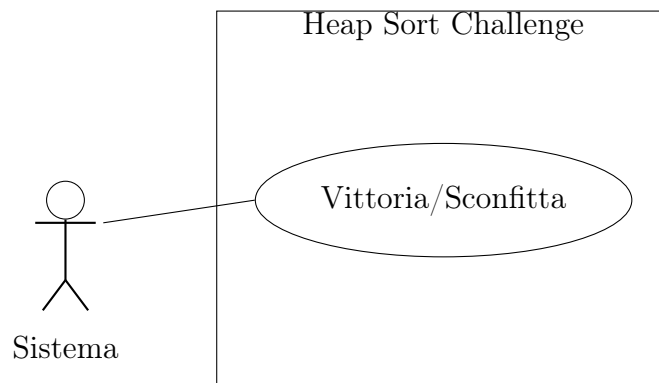


Figura 3.6: Diagramma dei Casi d'Uso per UC06 - Vittoria/Sconfitta

ID Caso d'Uso	UC06
Nome	Fine Partita (Vittoria/Sconfitta)
Attori Primari	Sistema (Trigger temporale o di stato)
Descrizione	Il sistema decreta la fine della partita quando l'array è completamente ordinato (Vittoria) o quando scade il tempo/mosse (Sconfitta).
Precondizioni	Partita in corso.

Postcondizioni	• Il gioco si ferma. • Viene mostrato un modal con il risultato. • Le statistiche vengono salvate (se previsto).
Scenario Principale	1. L'array diventa vuoto (tutti elementi estratti). 2. Il sistema rileva <code>heapSize == 0</code>. 3. Il sistema calcola il punteggio. 4. Il sistema mostra "Vittoria!".
Scenario Alternativo	Sconfitta: 1. Il tempo a disposizione scade. 2. Il sistema interrompe le interazioni. 3. Il sistema mostra "Game Over".

3.1.3 Matrice di Tracciabilità

La matrice di tracciabilità è uno strumento fondamentale per verificare la completezza dell'analisi. Essa mette in relazione i Requisiti Funzionali (RF) definiti nel Capitolo 2 con i Casi d'Uso (UC) progettati in questo capitolo, garantendo che ogni funzionalità richiesta sia effettivamente coperta da un flusso di interazione utente.

Case d'Uso	RF01	RF02	RF03	RF04	RF05
UC01 - Selezione	X				
UC02 - Inizio		X			
UC03 - Swap			X		
UC04 - Estrazione			X		
UC05 - Reset	X				
UC06 - Fine Gioco				X	X

3.2 Diagramma delle Attività

Il Diagramma delle Attività (Activity Diagram) è un diagramma comportamentale UML che descrive il flusso di lavoro (workflow) o di operazioni step-by-step di un sistema. A differenza dei Diagrammi dei Casi d'Uso che si concentrano su "chi fa cosa", gli Activity Diagram dettagliano "come" si susseguono le azioni, evidenziando decisioni, parallelismi e loop. È particolarmente utile per modellare la logica algoritmica e le regole di transizione di stato del gioco.

3.2.1 Attività Principali

Nel contesto di "Heap Sort Challenge", il flusso principale modella l'interazione ciclica tra Utente e Sistema durante una partita. Il diagramma seguente illustra il ciclo di vita di una sessione di gioco, dalla selezione della modalità fino alla proclamazione del vincitore.

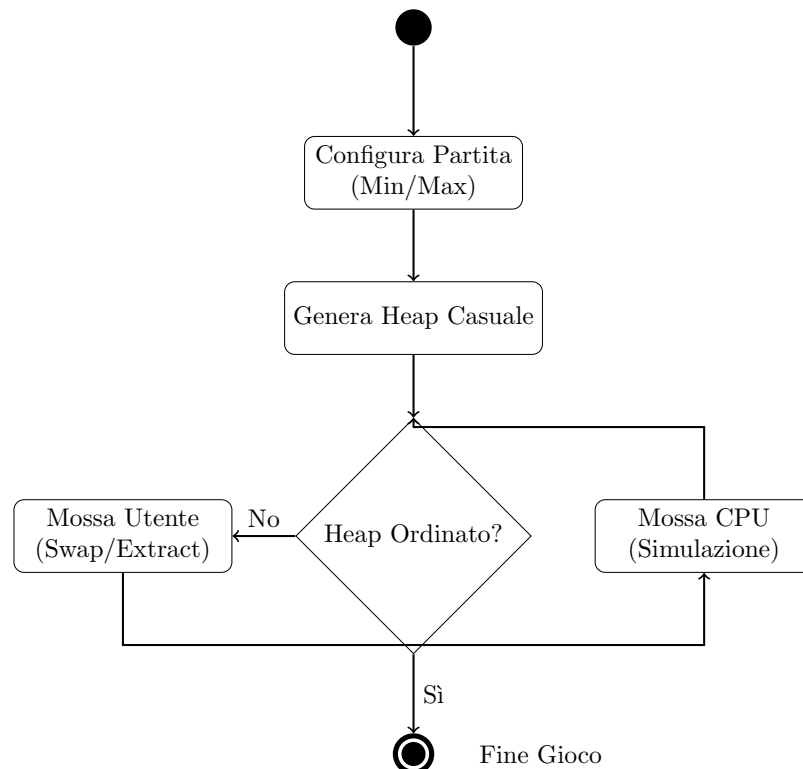


Figura 3.7: Diagramma delle Attività - Flusso di Gioco

Il flusso inizia con il **nodo iniziale** (cerchio pieno). L'utente configura la modalità e il sistema inizializza l'ambiente. Successivamente si entra nel **loop di gioco** (Decision Diamond): finché l'heap non è completamente ordinato, il controllo passa alternativamente all'Utente e alla CPU. Quando la condizione `heapSize == 0` è soddisfatta, il flusso termina nel **nodo finale** (cerchio bordato).

3.2.2 Dettaglio del Flusso Principale (Workflow)

Per garantire una corretta esecuzione dell'algoritmo e un'esperienza utente fluida, l'applicazione implementa il seguente workflow dettagliato:

1. Fase di Configurazione (Setup)

- L'utente seleziona la modalità di ordinamento desiderata (*Min-Heap* o *Max-Heap*) tramite l'interfaccia.
- Il sistema inibisce i controlli di gioco finché non viene premuto il pulsante di avvio.

2. Fase di Inizializzazione

- Alla pressione di "Start Game", il sistema genera un array di 15 numeri interi pseudo-casuali (range 1-99).
- Viene costruita la struttura ad albero binario visuale corrispondente all'array disordinato.
- Il timer di gioco viene avviato.

3. Core Loop (Ciclo di Gioco)

- *Turno Utente*: L'utente deve identificare e correggere le violazioni della proprietà di heap. Seleziona un nodo padre e un figlio; se lo scambio è valido (cioè rispetta l'algoritmo), il sistema anima lo spostamento nodale.
- *Validazione Continua*: Dopo ogni mossa, il sistema verifica se la radice è un estremo valido (minimo o massimo globale). In caso affermativo, la radice viene estratta automaticamente e spostata nella lista ordinata.
- *Turno CPU*: Se la modalità competitiva è attiva, la CPU esegue la sua mossa immediatamente dopo l'utente, simulando un "tempo di ragionamento" per realismo.

4. Fase di Terminazione

- Il ciclo continua finché l'heap non è vuoto (`heapSize = 0`).
- Il sistema confronta le metriche di performance (numero di mosse) tra Utente e CPU.
- Viene mostrato un modale di riepilogo con l'esito della partita (Vittoria/Sconfitta) e le statistiche dettagliate.

3.3 Diagramma delle Classi

Il Diagramma delle Classi (Class Diagram) è un diagramma strutturale UML che descrive l'architettura del sistema rappresentando le classi, i loro attributi, i metodi e le relazioni tra oggetti. È fondamentale per la progettazione orientata agli oggetti (OOP) in quanto mappa direttamente l'implementazione del codice, fungendo da "blueprint" per lo sviluppo software.

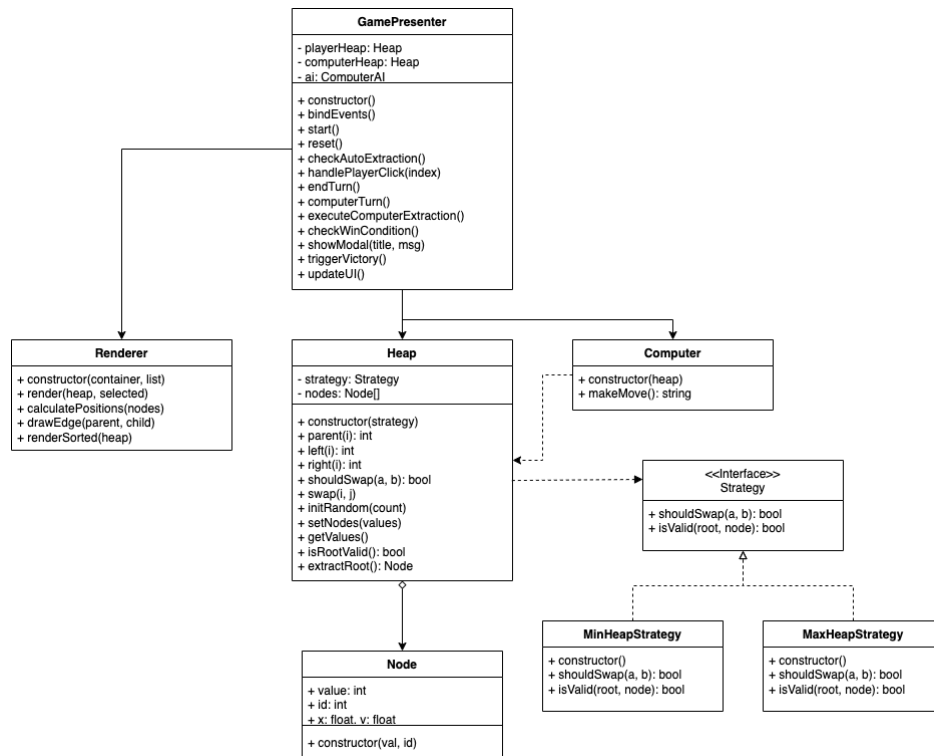


Figura 3.8: Diagramma delle Classi - Architettura MVP

Nel dettaglio dell'applicazione "Heap Sort Challenge", il diagramma evidenzia l'adozione del pattern architetturale **Model-View-Presenter (MVP)**

per separare la logica di business dalla rappresentazione visiva. Di seguito vengono analizzate in profondità le componenti principali:

3.3.1 GamePresenter (Presenter)

La classe `GamePresenter` funge da "orchestratore" dell'intera applicazione. Situata nel layer di presentazione, essa media le comunicazioni tra il modello dati (Heap) e la vista (Renderer), garantendo che la logica di gioco rimanga disaccoppiata dall'interfaccia utente.

Responsabilità Principali

- **Gestione del Ciclo di Vita:** Inizializza le istanze di gioco (`start()`), gestisce il reset (`reset()`) e monitora lo stato della partita (`isPlaying`, `isComputerTurn`).
- **Gestione Eventi:** Tramite il metodo `bindEvents()`, intercetta le azioni dell'utente (click sui nodi, pulsanti di controllo) e le traduce in comandi per il modello.
- **Coordinamento Turni:** Alterna i turni tra Utente e CPU. Quando l'utente completa una mossa, il presenter invoca `computerTurn()` dopo un delay artificiale per migliorare l'esperienza utente.
- **Validazione e Feedback:** Verifica se le mosse dell'utente sono lecite (es. scambio padre-figlio) e fornisce feedback visivo immediato tramite il Renderer.

Metodi Chiave

`start()` Configura una nuova partita istanziando gli Heap con la strategia selezionata (Min o Max). Genera i dati casuali e avvia il timer.

`handlePlayerClick(index)` Gestisce la logica di selezione multipla. Mantiene un array `selectedIndices`; quando due nodi sono selezionati, verifica la relazione di parentela e tenta lo scambio.

`checkAutoExtraction()` Verifica se la radice dell'heap soddisfa la proprietà di ordinamento. In caso positivo, scatena l'estrazione automatica e incrementa il contatore delle mosse.

`checkWinCondition()` Confronta la dimensione degli heap di giocatore e computer. Se uno dei due è vuoto, determina il vincitore basandosi sul numero di mosse e mostra il modale di fine partita.

3.3.2 Heap (Model)

La classe `Heap` incapsula la struttura dati e la logica algoritmica. È il cuore matematico dell'applicazione, responsabile del mantenimento della "Heap Property".

Design Pattern: Strategy

Per supportare sia Min-Heap che Max-Heap senza duplicare il codice, la classe `Heap` utilizza il **Strategy Pattern**. Al momento dell'istanziatura, riceve un oggetto che implementa l'interfaccia `Strategy` (con metodi `shouldSwap` e `isValid`). Questo permette all'Heap di delegare la logica di confronto, rimanendo agnostico rispetto al tipo di ordinamento specifico.

Attributi e Struttura

- **nodes**: Un array dinamico di oggetti `Node`. Rappresenta l'albero binario completo, dove per un nodo all'indice i , i figli si trovano a $2i + 1$ e $2i + 2$.
- **strategy**: Riferimento all'istanza della strategia corrente (`MinHeapStrategy` o `MaxHeapStrategy`).
- **sorted**: Array che raccoglie gli elementi mano a mano che vengono estratti dalla radice, formando la lista ordinata finale.

Metodi Chiave

`initRandom(count)` Popola l'heap con N numeri interi unici generati pseudocasualmente. Garantisce che non ci siano duplicati per semplificare l'apprendimento.

`swap(i, j)` Esegue lo scambio fisico di due nodi nell'array interno. È l'operazione atomica fondamentale dell'algoritmo.

`isRootValid()` Scansiona linearmente l'heap per verificare se la radice è l'estremo globale (minimo o massimo) rispetto a tutti gli altri nodi. Utilizza `strategy.isValid()`.

`extractRoot()` Rimuove la radice, sposta l'ultimo elemento in cima (o gestisce il caso di heap vuoto) e restituisce il nodo estratto per l'inserimento nella lista ordinata.

3.3.3 Renderer (View)

La classe `Renderer` è responsabile della rappresentazione grafica dello stato del gioco. Disaccoppiata dalla logica di business, si limita a riflettere visivamente i dati forniti dal Model, gestendo il Document Object Model (DOM) del browser.

Rendering Dinamico

Il rendering avviene in due fasi: calcolo posizionale e disegno.

- **Algoritmo di Posizionamento:** Per rappresentare l'heap come un albero binario bilanciato, il renderer calcola le coordinate (x, y) di ogni nodo. La larghezza disponibile viene suddivisa ricorsivamente in base alla profondità del nodo, garantendo che l'albero si adatti dinamicamente al container HTML.
- **Gestione DOM:** Utilizza elementi HTML standard (div) per disegnare nodi e archi. Ogni volta che lo stato cambia (es. dopo uno swap), il renderer ridisegna le connessioni ma ottimizza l'aggiornamento dei nodi esistenti per permettere animazioni CSS fluide (transizioni di posizione).

Metodi Chiave

`render(heap, selectedIndices)` Metodo principale invocato dal Presenter. Riceve l'oggetto Heap completo e la lista degli indici selezionati.

`calculatePositions(nodes)` Assegna le coordinate spaziali ad ogni nodo. Il livello L determina la coordinata Y , mentre la posizione nell'array determina l'offset X .

`drawEdge(parent, child)` Utilizza CSS transforms per tracciare una linea di connessione tra due nodi, aggiornandola dinamicamente durante le animazioni.

`renderSorted(heap)` Visualizza separatamente l'array `sorted`, mostrando all'utente i "frutti" del lavoro di ordinamento (es. i numeri estratti in ordine crescente o decrescente).

3.3.4 Computer

La classe **Computer** implementa la logica dell'avversario. Nonostante sia un'applicazione educativa, il computer deve simulare un comportamento "competente" per fornire una sfida.

Logica Decisionale

Il computer opera su una propria istanza di **Heap**, speculare a quella del giocatore all'inizio.

1. **Priorità all'Estrazione:** Il metodo `makeMove()` controlla prima di tutto se la radice è valida (`isRootValid()`). Se lo è, il computer "attende" il prossimo ciclo per estrarre, massimizzando il punteggio.
2. **Strategia di Correzione:** Se la radice non è pronta, il computer cerca attivamente violazioni della proprietà di heap. Scansiona l'albero per trovare nodi che violano la regola rispetto ai propri figli e propone uno scambio correttivo.
3. **Simulazione Umana:** Introduce ritardi artificiali per rendere le mosse visibili e comprensibili all'utente, evitando che il computer completi il gioco istantaneamente.

3.3.5 Node

La classe **Node** è una struttura dati semplice (POJO - Plain Old JavaScript Object) che funge da contenitore di informazioni.

- **value:** Il valore numerico intero da ordinare.
- **id:** Un identificativo univoco per tracciare l'elemento nel DOM durante i ri-ordinamenti, essenziale per le animazioni (chiave di riconciliazione).
- **x, y:** Coordinate calcolate dal **Renderer**, memorizzate nel nodo per evitare ricalcoli costanti durante un singolo frame di rendering.

3.3.6 Interfaccia Strategy

Il pattern **Strategy** è cruciale per la flessibilità del gioco.

- **MinHeapStrategy:** Implementa `shouldSwap(a, b)` restituendo `true` se $a > b$ (il padre è maggiore del figlio, violazione Min-Heap). Implementa `isValid` verificando $parent \leq child$.

- **MaxHeapStrategy:** Inverte la logica, restituendo `shouldSwap = true` se $a < b$.

Questa astrazione permette di cambiare la modalità di gioco (dal "Min Game" al "Max Game") iniettando semplicemente una diversa istanza di strategia nel costruttore dell'Heap, senza modificare una riga della logica centrale dell'Heap stesso.

3.4 Diagramma di Sequenza

Il Diagramma di Sequenza (Sequence Diagram) è un diagramma di interazione UML che illustra come gli oggetti collaborano in un determinato scenario, ponendo l'accento sull'ordine temporale dei messaggi scambiati. A differenza del Diagramma delle Classi (statico) o del Diagramma delle Attività (flusso di lavoro), il Diagramma di Sequenza cattura la dinamica "runtime" del sistema.

Nel contesto di questa applicazione, l'adozione dei Diagrammi di Sequenza è fondamentale per documentare e comprendere due aspetti critici che non emergono dagli altri diagrammi:

- **Il Flusso MVP:** Visualizza chiaramente il percorso "round-trip" di un'azione. Ad esempio, un click dell'utente (View) viene intercettato dal Presenter, che interroga il Model, applica la logica di business, e infine comanda alla View di aggiornarsi. Questo assicura che il principio di "Separation of Concerns" sia rispettato rigorosamente.
- **Gestione Temporale e Animazioni:** L'applicazione utilizza transizioni visive per mostrare gli scambi e introduce ritardi funzionali durante il turno dell'avversario per migliorare la comprensione dell'algoritmo. Il diagramma esplicita come il sistema coordini questi eventi asincroni, assicurando che l'aggiornamento dello stato logico avvenga solo al termine delle animazioni, garantendo così la coerenza visiva.

L'utilizzo di questo diagramma ci permette di verificare la correttezza della logica di coordinamento, specialmente nelle fasi più complesse come l'estrazione ricorsiva della radice o il turno automatizzato del computer.

3.4.1 Sequenza Principale del Gioco

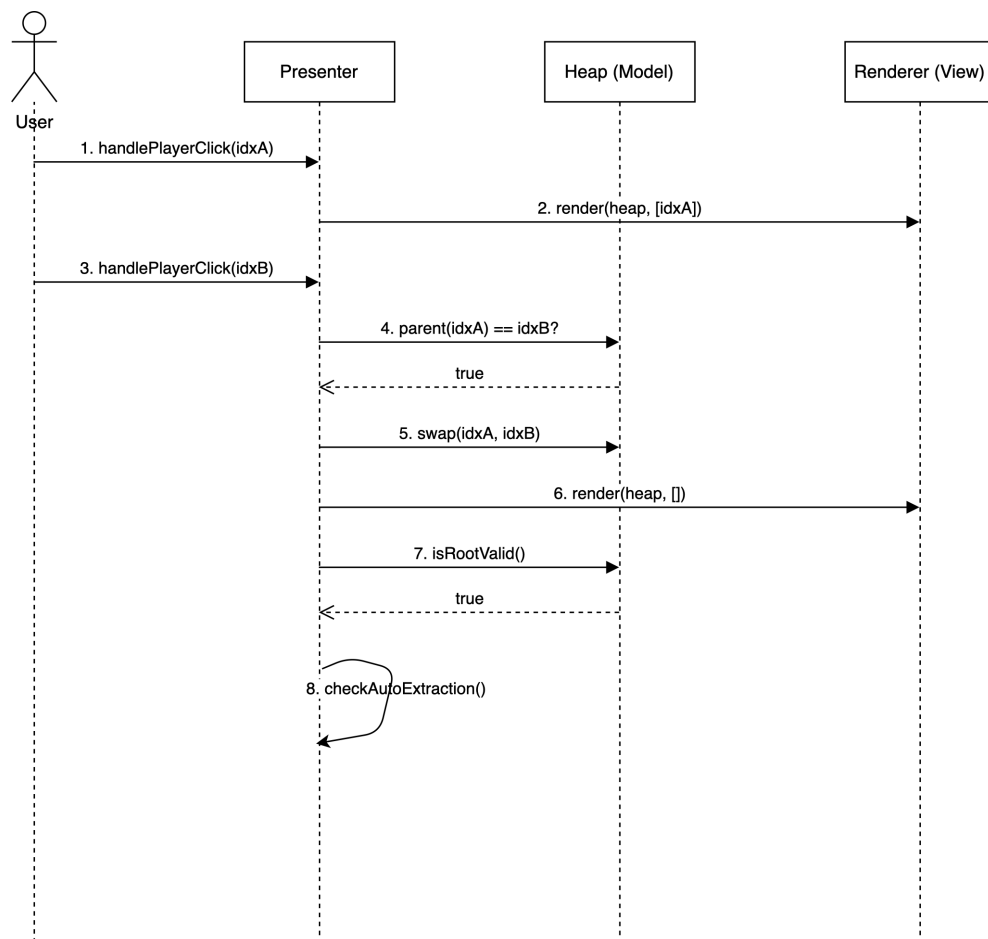


Figura 3.9: Diagramma di Sequenza - Scenario: Selezione e Swap Utente

Il diagramma di sequenza in Figura 3.9 descrive dettagliatamente il flusso di interazione durante una mossa di gioco, specificamente lo scambio (**swap**) tra due nodi deciso dall'utente. Questa operazione, apparentemente semplice, innesca una catena complessa di eventi che attraversa tutti i livelli dell'architettura MVP.

Di seguito analizziamo nel dettaglio le fasi che compongono questo scenario:

3.4.1.1 1. Gestione degli Eventi e Selezione (View -> Presenter)

Tutto ha inizio nel livello della Vista (**Renderer**). L'applicazione implementa il pattern **Event Delegation**: invece di attaccare un listener a ogni singolo nodo (che sarebbe dispendioso in termini di memoria), un unico event listener è collegato al contenitore padre dell'albero. Quando l'utente clicca su un nodo (identificato come `idxA`):

- L'evento `click` risale il DOM fino al contenitore.
- Il **Renderer** estrae l'ID del nodo target dall'attributo `data-index`.
- Il controllo viene passato al **GamePresenter** tramite il metodo `handlePlayerClick(idxA)`.

Il Presenter mantiene lo stato della selezione interna. Se è il primo click, il nodo viene semplicemente marcato come "selezionato" e il Renderer viene istruito di evidenziarlo visivamente (cambio di colore e spessore del bordo). Il diagramma si concentra sul caso del **secondo click** (`idxB`), che innesca il tentativo di mossa.

3.4.1.2 2. Validazione Logica (Presenter -> Model)

Prima di modificare qualsiasi dato, il sistema deve garantire l'integrità delle regole del gioco. Il **Presenter** interroga il **Model** (**Heap**) per validare la mossa:

1. **Adiacenza**: I due nodi devono essere adiacenti nell'albero (padre-figlio). La formula matematica $parent = \lfloor (child - 1) / 2 \rfloor$ permette una verifica istantanea $O(1)$.
2. **Proprietà di Heap**: Non basta che i nodi siano vicini; il loro scambio deve essere "utile".
 - In un *Max-Heap*, se il figlio è maggiore del padre ($A[child] > A[parent]$), lo scambio è necessario.
 - In un *Min-Heap*, se il figlio è minore del padre ($A[child] < A[parent]$), lo scambio è necessario.

Questa logica di validazione è incapsulata nel metodo `shouldSwap()` della Strategy attiva. Se la validazione fallisce, il Presenter innesca un feedback di errore (animazione "shake") e resetta la selezione senza toccare il Modello.

3.4.1.3 3. Manipolazione dei Dati e Coerenza (Model)

Se la validazione ha successo (il percorso marcato come `true` nel diagramma), il Presenter invoca il comando `swap(idxA, idxB)` sul Modello. Questa è un'operazione critica che modifica lo stato persistente dell'applicazione:

- L'array sottostante viene aggiornato: i valori alle posizioni A e B vengono scambiati.
- Gli oggetti `Node` mantengono i loro ID univoci ma cambiano posizione nell'array. Questo è fondamentale per permettere al framework di rendering di tracciare l'identità degli elementi (Object Identity) e animarli correttamente.

3.4.1.4 4. Riconciliazione della Vista (Presenter -> View)

Una volta mutato il Modello, la Vista deve essere sincronizzata. Invece di ridisegnare tutto l'albero da zero (che causerebbe un "flash" sgradevole e la perdita del contesto visivo), il sistema adotta un approccio differenziale:

- Il Presenter chiama `render(heap)`.
- Il `Renderer` calcola le nuove coordinate (x, y) target per ogni nodo.
- Poiché i nodi DOM hanno un ID stabile, il browser può interpolare la posizione vecchia e quella nuova.
- Viene applicata una transizione CSS (`transition: transform 0.5s`).

Durante questa fase di animazione, l'input utente viene temporaneamente disabilitato per evitare "Race Conditions" (es. l'utente clicca su un nodo che si sta muovendo).

3.4.1.5 5. Verifica Post-Mossa e Automazione (Chain Reaction)

Il diagramma termina con una fase di verifica globale (`isRootValid` e `checkAutoExtraction`). Dopo uno scambio, la nuova configurazione potrebbe aver sbloccato nuove possibilità:

- La radice potrebbe essere diventata il massimo/minimo assoluto. In tal caso, il sistema deve procedere all'estrazione automatica.
- Potrebbero essersi create nuove violazioni a cascata nei sotto-alberi (anche se nel caso del giocatore singolo, le mosse sono puntuali).

Se la radice è valida, il Presenter scatena l'evento di estrazione, che a sua volta genererà un nuovo ciclo di aggiornamenti (rimozione nodo, spostamento ultimo elemento, nuova fase di heapify), dimostrando la natura ricorsiva e ciclica dell'algoritmo Heap Sort.

In sintesi, la Figura 3.9 non mostra solo uno "spostamento di numeri", ma la coreografia precisa tra input asincrono dell'utente, validazione sincrona della logica matematica e rendering differito per l'esperienza utente.

3.4.2 Interazione tra Classi - Turno Computer

Il diagramma seguente (Figura 3.10) illustra le interazioni tra le componenti del sistema durante il turno del Computer.

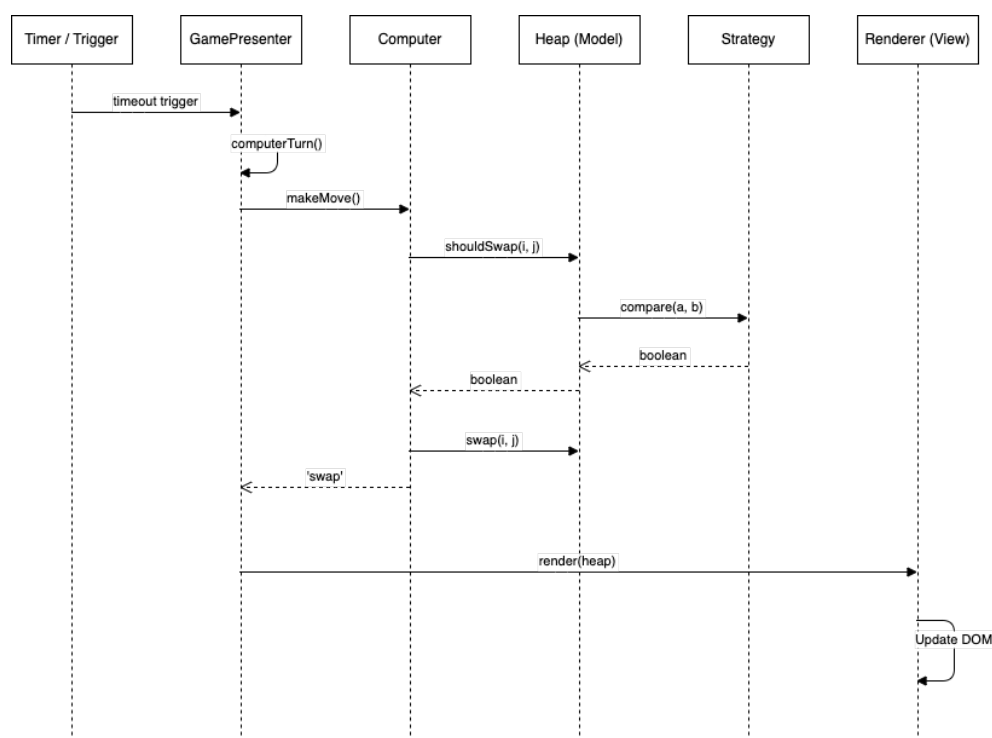


Figura 3.10: Diagramma di Sequenza - Comunicazione tra Classi (Turno Computer)

Il flusso di esecuzione delle operazioni durante il turno del Computer, rappresentato nella Figura 3.10, è un esempio paradigmatico di come un'architettura Model-View-Presenter possa gestire la logica asincrona e algoritmica senza compromettere la fluidità dell'interfaccia. Di seguito viene presentata un'analisi tecnica approfondita, suddivisa in fasi logiche che mappano gli 11 passaggi del diagramma.

3.4.2.1 1. Orchestrazione Temporale e Innesco (Timer -> Presenter)

A differenza del turno del giocatore, che è guidato da input diretti (click), il turno del computer è un processo **time-driven**. Tutto ha inizio con un'interazione tra un timer asincrono (`setTimeout`) e il `GamePresenter`:

- **Decoupling:** L'invocazione di `computerTurn()` avviene dopo un ritardo variabile (tipicamente 1000-1500ms). Questo è fondamentale per la User Experience, poiché permette all'utente di "vedere" il computer che pensa, evitando un'esecuzione istantanea che risulterebbe confusionaria.
- **State Management:** Il Presenter imposta internamente il flag `isComputerTurn` a `true`. Questo blocca temporaneamente gli input del giocatore, prevenendo "Race Conditions" dove l'utente e il computer potrebbero tentare di modificare lo stesso Heap contemporaneamente.

3.4.2.2 2. Analisi Strategica e Scansione del Modello (Computer -> Heap)

Ricevuto il comando dal Presenter (`makeMove()`), la classe `Computer` inizia la fase di analisi. Il computer non "vede" l'interfaccia grafica; esso interagisce esclusivamente con l'interfaccia pubblica del Modello (`Heap`):

- **Ricerca di Violazioni:** Il computer scansiona l'array dello heap cercando la prima coppia di nodi (padre-figlio) che viola la proprietà fondamentale dell'algoritmo (es. un padre più piccolo di un figlio in un Max-Heap).
- **Agnosticismo dei Dati:** Il computer non sa se sta giocando con un Min-Heap o un Max-Heap. Esso si limita a chiedere allo Heap: `shouldSwap(parent, child)`. Questa astrazione è il primo livello di difesa contro la duplicazione del codice.

3.4.2.3 3. Il Pattern Strategy in Azione (Model -> Strategy)

Il diagramma evidenzia il ruolo cruciale del **Pattern Strategy** (passaggi 5 e 6). Lo `Heap` riceve la richiesta di validazione dal `Computer` e la delega immediatamente alla strategia attiva:

- **Delegazione Pura:** Lo `Heap` possiede un riferimento a un'interfaccia `Strategy`. Se la strategia è `MinHeapStrategy`, il confronto restituirà `true` se il figlio è minore del padre. Se è `MaxHeapStrategy`, la logica sarà opposta.
- **Separazione delle Preoccupazioni:** Questa fase dimostra come la logica matematica pura sia isolata dalla struttura dati. Lo `Heap` gestisce l'albero (posizioni, indici), mentre la `Strategy` gestisce la verità matematica (confronto).

3.4.2.4 4. Esecuzione della Mutazione e Integrità del Modello (Model Execution)

Una volta ottenuta la conferma (`boolean: true`), il Computer passa all'azione (passaggio 8). Invocando `swap(i, j)`, il computer modifica lo stato interno del sistema:

- **Atomicità:** Lo scambio dei valori nell'array è un'operazione atomica che garantisce che il Modello sia sempre in uno stato consistente prima di passare alla fase successiva.
- **Preservazione dell'Identità:** Nonostante i valori vengano scambiati, gli oggetti `Node` coinvolti mantengono il loro ID univoco. Questo permetterà successivamente al `Renderer` di tracciare il movimento "fisico" del nodo sullo schermo invece di limitarsi a cambiare un testo.

3.4.2.5 5. Feedback Loop e Ritorno del Controllo (Computer -> Presenter)

Terminata la mossa, il Computer restituisce un segnale al Presenter (passaggio 9). Questo passaggio è vitale per la comunicazione inter-livello:

- **Signal Handling:** Il ritorno della stringa `'swap'` comunica al Presenter che un'azione è stata effettivamente compiuta e che è necessario un aggiornamento visivo.
- **Analisi Post-Mossa:** Se il computer non avesse trovato mosse valide, avrebbe restituito un valore diverso (es. `'no-move'`), indicando al Presenter che il turno del computer è terminato o che è necessaria un'estrazione.

3.4.2.6 6. Sincronizzazione della Vista e Rendering Differenziale (View Phase)

L'ultima fase del diagramma descrive il passaggio dal dato logico all'immagine visiva:

1. **Comando di Rendering:** Il Presenter chiama `render(heap)`. È importante notare che il Presenter passa l'intero stato dello Heap alla Vista, mantenendo la Vista "stupida": essa non deve calcolare cosa è successo, ma solo "dipingere" lo stato attuale.
2. **Ottimizzazione del DOM:** Il `Renderer` calcola le nuove coordinate (x, y) per i nodi modificati. Grazie alla persistenza degli ID degli

elementi DOM, il CSS può applicare transizioni fluide tra le vecchie coordinate e le nuove, creando l'illusione di un movimento fluido che aiuta l'apprendimento dell'utente.

3.4.2.7 7. Considerazioni Finali sul Flusso Computer

In conclusione, la densità di messaggi mostrata in Figura 3.10 riflette la robustezza dell'architettura. Suddividere il turno del computer in 11 passaggi permette di:

- **Testability:** Ogni passaggio (validazione, scambio, rendering) può essere testato unitariamente.
- **Scalability:** È possibile aggiungere nuove strategie di gioco o nuovi tipi di computer (es. un computer "difficile") semplicemente implementando nuove sottoclassi di **Computer** o **Strategy**, senza mai toccare il codice del **Presenter** o del **Renderer**.

Il risultato finale è un sistema dove la "macchina" gioca contro l'utente seguendo le stesse rigide regole matematiche, garantendo un'esperienza educativa coerente e priva di errori logici.